🌿 TOPIC 11.0: TRIE FOUNDATION & REAL-WORLD USE CASES

🔗 See Topic 10 → Trees & BST Masterclass

🎯 **GOAL:** Master Trie (Prefix Tree) data structures, understand shared prefix node re-use, solve Prefix Search, Autocomplete Suggestions, Word Search II (DFS Pruning), and Bitwise Maximum XOR!

🧰 REUSABLE TRIE UTILITIES & NODE CLASS

```
class TrieNode {
    TrieNode[] children = new TrieNode[26];
    boolean isEnd = false;
}
void insert(TrieNode root, String word) {
    TrieNode curr = root;
    for (char c : word.toCharArray()) {
        int idx = c - 'a';
        if (curr.children[idx] == null) curr.children[idx] = new TrieNode();
        curr = curr.children[idx];
    }
    curr.isEnd = true;
}
```

💡 **AHA MOMENT #1: WHY TRIE OVER HASHMAP OR BST?**
A HashMap gives $O(L)$ exact lookup (where L is word length), BUT finding all words with prefix `"cat"` requires scanning ALL N keys in $O(N \cdot L)$ time!
A **Trie (Prefix Tree)** shares common prefixes across words. Searching for prefix `"cat"` takes ONLY **$O(L)$ time**, independent of total dictionary size N!

1 REAL-WORLD FAANG APPLICATIONS

- **Google Search Autocomplete:** Suggesting top queries matching typed prefix.
- **Spell Checkers & T9 Texting:** Finding valid dictionary words matching prefix.
- **IP Routing Tables (Longest Prefix Match):** Forwarding packets to destination subnets.
- **Genome Sequence Matching:** Matching DNA prefix patterns.

2 TRIE VS HASHMAP VS BST MATRIX

Feature	Trie (Prefix Tree)	HashMap	Binary Search Tree
Exact Word Search	$O(L)$	$O(L)$	$O(L \log N)$
Prefix Search (`startsWith`)	✅ **$O(L)$**	❌ $O(N \cdot L)$	⚠️ $O(L \log N)$
Space Complexity	Shared Prefix (Low)	No Sharing (High)	Pointers per node

📌 **KEY TAKEAWAY:** Reach for a Trie whenever a problem mentions "Prefix", "Autocomplete", "Dictionary Search", or "Maximum XOR"!

🛠️ TOPIC 11.1: TRIE CONSTRUCTION & MEMORY LAYOUT

💡 **AHA MOMENT #2: TrieNode STRUCTURE & CHILDREN REPRESENTATION**
Each node in a Trie represents a single character link. Nodes do NOT store their own character value; instead, the character is implicit from the edge pointer index (`children[c - 'a']`)!

1 JAVA TRIENODE CLASS IMPLEMENTATION

```
public class TrieNode {
    // Array of 26 child pointers for 'a' through 'z'
    public TrieNode[] children = new TrieNode[26];

    // Marks if a valid word ends at this node
    public boolean isEnd = false;

    // Optional metadata counters:
    public int prefixCount = 0; // Number of words sharing this prefix
    public int wordCount = 0;   // Number of exact word completions
}
```

2 ARRAY[26] VS HASHMAP CHILDREN

Implementation	Pros	Cons	FAANG Verdict
`TrieNode[26]`	Instant $O(1)$ index access, 0 collision overhead	Wastes null memory for small alphabets	✅ **BEST for a-z**
`HashMap<Char, Node>`	Saves space for arbitrary Unicode / ASCII	HashMap object overhead & hash lookup	Use for generic Unicode

⚠️ **COMMON BUG:** Forgetting to subtract `'a'` when indexing `children[c - 'a']` causes `ArrayIndexOutOfBoundsException`!

🎨 TOPIC 11.2: CORE OPERATIONS & CHARACTER INSERTION VISUALIZER

💡 **AHA MOMENT #3: PREFIX SHARING & RE-USE IN ACTION**

When inserting `"cat"` followed by `"car"`, the characters `'c'` and `'a'` ALREADY EXIST in the Trie. The algorithm reuses existing nodes `(root) ➔ c ➔ a` and creates ONLY a single new node `'r'` for `"car"`!

🎨 STEP-BY-STEP TRIE GROWTH ANIMATION: INSERT "CAT", THEN INSERT "CAR"

1. INSERT "cat": Step 1 ('c'): (root) └── c Step 2 ('a'): (root) └── c └── a Step 3 ('t'): (root) └── c └── a └── t★ (isEnd = true) **2. NOW INSERT "car":** Reuses existing shared prefix 'c' ➔ 'a'! Step 1 ('c'): Reused existing node 'c' Step 2 ('a'): Reused existing node 'a' Step 3 ('r'): Created NEW node 'r★' (isEnd = true) **FINAL TRIE STRUCTURE:** (root) └── c (reused) └── a (reused) ├── t★ (word: "cat") └── r★ (word: "car", new node!)

📌 **KEY TAKEAWAY:** Space efficiency increases as dictionary size grows because thousands of words share common prefixes!

🔥 TOPIC 11.4: DFS + TRIE (WORD SEARCH II & PREFIX PRUNING)

💡 **AHA MOMENT #4: 1000x SPEEDUP VIA TRIE PREFIX PRUNING**
Standard Grid Backtracking searches for each word independently ($O(N \cdot M \cdot 4^L)$ TLE!).
By building a **Trie of all target words**, we explore the grid ONCE. If the current grid path does NOT exist in the Trie, we **prune the entire DFS branch immediately**!

📌 WORD SEARCH II PREFIX PRUNING FLOWCHART

Grid Cell (r, c) ➔ Step on character ➔ Check Trie node.children[c - 'a'] ├── If NULL ➔ PRUNE DFS BRANCH IMMEDIATELY! (Path is not a prefix of any word) └── If EXISTS ➔ Move to child Trie node ➔ If node.isEnd == true ➔ Add word to result & set isEnd=false!

📌 **OPTIMIZATION TRICK:** Store the entire string `node.word` directly inside the `TrieNode` to avoid `StringBuilder` append/delete string manipulation overhead!

TOPIC 11.5: BITWISE TRIE (MAXIMUM XOR OF TWO NUMBERS) PAGE 5 OF 11

💡 **AHA MOMENT #5: GREEDY OPPOSITE BIT SELECTION IN BITWISE TRIE**

XOR ($A \oplus B$) produces `1` when bits differ ($1 \oplus 0 = 1$) and `0` when bits match ($1 \oplus 1 = 0$).
To maximize XOR of number A, insert all numbers as **31-bit Binary Trees** (left branch = 0, right branch = 1). For each bit b of A from MSB (31) down to 0, **greedily pick the OPPOSITE bit `1 - b`** in the Trie!

🎨 **VISUALIZING 3-BIT BINARY TRIE FOR NUMBERS [3 (011), 5 (101)]**

```
(root) / \ [0] [1] (Bit 2 - MSB) \ / [1] [0] (Bit 1) / \ [1] [1] (Bit 0 - LSB) (val 3) (val 5) To maximize XOR for number 3 (011): Bit 2 (0): Pick opposite bit 1 ➔ Moves to node [1] Bit 1 (1): Pick opposite bit 0 ➔ Moves to node [0] Bit 0 (1): Opposite bit 0 missing, take 1 ➔ Moves to node [1] Max XOR = 3 ^ 5 = 6 (110)!
```

📌 **KEY TAKEAWAY:** A Bitwise Trie cuts Maximum XOR pair search from $O(N^2)$ brute-force to $O(32 \cdot N) = O(N)$ linear time!

📜 TOPIC 11.6 & 1-PAGE PRINTABLE TRIE CHEAT SHEET

🌳 TRIE INTERVIEW DECISION TREE (MERMAID)

📜 PRINTABLE TRIE & BITWISE TRIE CHEAT SHEET

Pattern Type	Target Problem	Trie Architecture	Time Complexity
Standard Prefix Tree	Implement Trie (LC 208)	`TrieNode[26]` + `isEnd`	Insert/Search: $O(L)$
Wildcard Search	Add & Search Words (LC 211)	Trie + DFS Backtracking on `'.'`	Search: $O(26^L)$ worst
Grid Backtracking	Word Search II (LC 212)	Trie + Grid DFS + Pruning	Time: $O(M \cdot N \cdot 4^L)$
Search Suggestions	Search Suggestions (LC 1268)	Trie + Top-3 List at nodes	Search: $O(L)$
Bitwise XOR	Max XOR Pair (LC 421)	Binary 31-bit Trie `[0, 1]`	Insert/Find: $O(32)$

🎓 **FAANG COACH TIP:** Print this page and review it 10 minutes before your technical interview!

🧩 EASY NEETCODE: IMPLEMENT TRIE (PREFIX TREE) PAGE 7 OF 11

1. LEETCODE 208 — IMPLEMENT TRIE (PREFIX TREE)

🟡 Medium ★★★★★ **Core Trie Operations** Amazon Google Meta

Problem:

Implement a Trie with

`insert`

,

`search`

, and

`startsWith`

methods.

```
class Trie {
    private class TrieNode {
        TrieNode[] children = new TrieNode[26];
        boolean isEnd = false;
    }
    private TrieNode root = new TrieNode();

    public void insert(String word) {
        TrieNode curr = root;
        for (char c : word.toCharArray()) {
            int idx = c - 'a';
            if (curr.children[idx] == null) curr.children[idx] = new
TrieNode();
            curr = curr.children[idx];
        }
        curr.isEnd = true;
    }

    public boolean search(String word) {
        TrieNode curr = find(word);
        return curr != null && curr.isEnd;
    }

    public boolean startsWith(String prefix) {
        return find(prefix) != null;
    }

    private TrieNode find(String str) {
        TrieNode curr = root;
        for (char c : str.toCharArray()) {
            int idx = c - 'a';
            if (curr.children[idx] == null) return null;
            curr = curr.children[idx];
        }
        return curr;
    }
}
```

💡 **AHA MOMENT (LC 208):** Helper method `find(str)` eliminates duplicate code between `search()` and `startsWith()`!

2. LEETCODE 211 — DESIGN ADD AND SEARCH WORDS DATA STRUCTURE

Medium ★★★★★ Trie + Wildcard DFS Amazon Meta

Problem:

Support `addWord(word)` and `search(word)` where `.` matches any character.

```
class WordDictionary {
    private class TrieNode {
        TrieNode[] children = new TrieNode[26];
        boolean isEnd = false;
    }
    private TrieNode root = new TrieNode();

    public void addWord(String word) {
        TrieNode curr = root;
        for (char c : word.toCharArray()) {
            int idx = c - 'a';
            if (curr.children[idx] == null) curr.children[idx] = new
TrieNode();
            curr = curr.children[idx];
        }
        curr.isEnd = true;
    }

    public boolean search(String word) {
        return dfs(word, 0, root);
    }
    private boolean dfs(String word, int idx, TrieNode curr) {
        if (curr == null) return false;
        if (idx == word.length()) return curr.isEnd;
        char c = word.charAt(idx);
        if (c == '.') {
            for (TrieNode child : curr.children) {
                if (child != null && dfs(word, idx + 1, child)) return
true;
            }
            return false;
        } else {
            return dfs(word, idx + 1, curr.children[c - 'a']);
        }
    }
}
```

3. LEETCODE 648 — REPLACE WORDS

Medium ★★★★☆ Shortest Root Prefix Uber

Problem:

Replace words in sentence with shortest matching root prefix in dictionary.

```
public String replaceWords(List<String> dictionary, String sentence) {
    TrieNode root = new TrieNode();
    for (String word : dictionary) {
        TrieNode curr = root;
        for (char c : word.toCharArray()) {
            int idx = c - 'a';
            if (curr.children[idx] == null) curr.children[idx] = new
TrieNode();
            curr = curr.children[idx];
        }
        curr.isEnd = true;
    }
    StringBuilder res = new StringBuilder();
    for (String word : sentence.split(" ")) {
        if (res.length() > 0) res.append(" ");
        res.append(findRoot(word, root));
    }
    return res.toString();
}
private String findRoot(String word, TrieNode root) {
    TrieNode curr = root;
    StringBuilder prefix = new StringBuilder();
    for (char c : word.toCharArray()) {
        int idx = c - 'a';
        if (curr.children[idx] == null) break;
```

💡 MEDIUM NEETCODE: SEARCH SUGGESTIONS & MAP SUM PAIRS

4. LEETCODE 1268 — SEARCH SUGGESTIONS SYSTEM

🟡 Medium ★★★★★ **Trie Top-3 Suggestion List** Amazon

Problem:

Return top 3 lexicographically smallest matching products after typing each character of search word.

```
class Solution {
    class TrieNode {
        TrieNode[] children = new TrieNode[26];
        List<String> suggestions = new ArrayList<>();
    }
    public List<List<String>> suggestedProducts(String[] products,
String searchWord) {
        Arrays.sort(products); // Sorted for lexicographical order!
        TrieNode root = new TrieNode();
        for (String p : products) {
            TrieNode curr = root;
            for (char c : p.toCharArray()) {
                int idx = c - 'a';
                if (curr.children[idx] == null) curr.children[idx] = new
TrieNode();
                curr = curr.children[idx];
                if (curr.suggestions.size() < 3)
curr.suggestions.add(p); // Store Top 3!
            }
        }
        List<List<String>> res = new ArrayList<>();
        TrieNode curr = root;
        for (char c : searchWord.toCharArray()) {
            if (curr != null) curr = curr.children[c - 'a'];
            res.add(curr != null ? curr.suggestions : new ArrayList<>
());
        }
        return res;
    }
}
```

5. LEETCODE 677 — MAP SUM PAIRS

🟡 Medium ★★★★★ **Prefix Sum Accumulation** Google

Problem:

Sum values of all key-value pairs whose key starts with prefix.

```
class MapSum {
    private class TrieNode {
        TrieNode[] children = new TrieNode[26];
        int val = 0;
    }
    private TrieNode root = new TrieNode();
    private Map<String, Integer> map = new HashMap<>();

    public void insert(String key, int val) {
        int diff = val - map.getOrDefault(key, 0);
        map.put(key, val);
        TrieNode curr = root;
        for (char c : key.toCharArray()) {
            int idx = c - 'a';
            if (curr.children[idx] == null) curr.children[idx] = new
TrieNode();
            curr = curr.children[idx];
            curr.val += diff; // Accumulate delta sum!
        }
    }

    public int sum(String prefix) {
        TrieNode curr = root;
        for (char c : prefix.toCharArray()) {
            int idx = c - 'a';
            if (curr.children[idx] == null) return 0;
            curr = curr.children[idx];
        }
        return curr.val; // Instant O(L) sum!
    }
}
```

6. LEETCODE 212 — WORD SEARCH II

Hard ★★★★★ Trie + Grid DFS Pruning Amazon Meta Google

Problem:

Find all dictionary words present on a 2D board.

```
class Solution {
    class TrieNode {
        TrieNode[] children = new TrieNode[26];
        String word = null;
    }
    public List<String> findWords(char[][] board, String[] words) {
        TrieNode root = new TrieNode();
        for (String w : words) {
            TrieNode curr = root;
            for (char c : w.toCharArray()) {
                int idx = c - 'a';
                if (curr.children[idx] == null) curr.children[idx] = new
TrieNode();
                curr = curr.children[idx];
            }
            curr.word = w; // Store full word at leaf!
        }
        List<String> res = new ArrayList<>();
        for (int r = 0; r < board.length; r++) {
            for (int c = 0; c < board[0].length; c++) dfs(board, r, c,
root, res);
        }
        return res;
    }
    private void dfs(char[][] board, int r, int c, TrieNode node,
List<String> res) {
        char ch = board[r][c];
        if (ch == '#' || node.children[ch - 'a'] == null) return; //
Prune branch!
        node = node.children[ch - 'a'];
        if (node.word != null) { res.add(node.word); node.word = null; }
// Avoid dups
        board[r][c] = '#'; // Mark visited
        int[][] dirs = {{0,1},{1,0},{0,-1},{-1,0}};
        for (int[] d : dirs) {
            int nr = r + d[0], nc = c + d[1];
            if (nr >= 0 && nr < board.length && nc >= 0 && nc <
board[0].length) dfs(board, nr, nc, node, res);
        }
        board[r][c] = ch; // Backtrack
    }
}
```

7. LEETCODE 421 — MAXIMUM XOR OF TWO NUMBERS IN AN ARRAY

Hard ★★★★★ Bitwise 31-Bit Binary Trie Google

Problem:

Maximum result of $A \oplus B$ for two numbers in array in $O(N)$ time.

```
public int findMaximumXOR(int[] nums) {
    class Node { Node[] children = new Node[2]; }
    Node root = new Node();
    for (int num : nums) {
        Node curr = root;
        for (int i = 31; i >= 0; i--) {
            int bit = (num >> i) & 1;
            if (curr.children[bit] == null) curr.children[bit] = new
Node();
            curr = curr.children[bit];
        }
    }
    int maxXor = 0;
    for (int num : nums) {
        Node curr = root; int currXor = 0;
        for (int i = 31; i >= 0; i--) {
            int bit = (num >> i) & 1, opp = 1 - bit;
            if (curr.children[opp] != null) {
                currXor |= (1 << i); curr = curr.children[opp]; //
```

💎 IF YOU REMEMBER ONLY ONE PAGE... MASTER REVISION SHEET

⭐ THE 4 GOLDEN LAWS OF TRIES

1. **Array vs Map Children:** Use `TrieNode[26]` for lowercase english (`a-z`). Use `HashMap` for generic Unicode.
2. **Shared Prefix Efficiency:** Insert/Search time is $O(L)$ (where L is word length), independent of total words N.
3. **DFS Grid Pruning (Word Search II):** Pass Trie node along grid DFS. Prune immediately if `curr.children[c - 'a'] == null`!
4. **Bitwise Binary Trie:** To maximize XOR $A \oplus B$, greedily pick the opposite bit `1 - bit` at each of the 32 bit positions!

⛔ WHEN TRIE FAILS (DO NOT USE!)

- **Exact Key Lookups without Prefix Queries:** Standard key-value lookup ➔ Use **HashMap**!
- **Range Queries (e.g. Find values between X and Y):** Trie does not maintain binary ordering ➔ Use **TreeMap (BST)**!

📊 NEETCODE & BLIND 75 CONFIDENCE TRACKER

- Med LC 208 Implement Trie
- Med LC 211 Add & Search Words
- Med LC 648 Replace Words
- Med LC 1268 Search Suggestions
- Med LC 677 Map Sum Pairs
- Hard LC 212 Word Search II
- Hard LC 421 Maximum XOR

👉 **Next Master Topic: Topic 12 → Graphs Masterclass (BFS / DFS / Topological Sort)**

🎉 **TOPIC 11 TRIE MASTERCLASS COMPLETE!** Turn the page to **Topic 12: Graphs Masterclass**!